

DRIFT-AWARE EXPLAINABLE ANOMALY DETECTION FOR BEHAVIOURAL THREAT HUNTING

Zero-Day Detection — HOSTFUSE & HELD-OUT

A Project Report Submitted in Partial Fulfilment of the Requirements for
Bachelor of Technology in Computer Science & Engineering

Semester 7 — Final Year Project

Chapters 1–5: Introduction, Literature Review, Methodology, Design &
Modelling, Results

Submitted by: Deep (B — Detection Modeling)
Saharsh (A — Data & Capture)
Aditya (C — Trust & Risk)
Avinash (D — Adversarial & Delivery)

Guide: Individual Guide (Presentation 10–15 September)

Institution: Department of Computer Science & Engineering

Date: September 2026

CERTIFICATE

This is to certify that the project entitled **Drift-Aware Explainable Anomaly Detection for Behavioural Threat Hunting** is a bona fide work carried out by the team in partial fulfilment for the award of B.Tech. The report Chapters 1–4 have been prepared as per the documentation guideline (1.8 DOCUMENTATION) exemplified by the extracted samples (pp. 43–73 and pp. 9–22).

Signature of Guide _____
Date _____

DECLARATION

We declare that this report is our original work and has not been submitted elsewhere. References to prior work are cited. Datasets (CICIDS2017 GeneratedLabelledFlows, IDS2018, CTU-13) are used under CIC terms; `data/` and `venv/` are local-only and not committed. Model binaries `*.pt` are tracked for demo.

Deep (B)

Date: _____

ACKNOWLEDGEMENTS

We thank our guide, the CIC for datasets, and the open-source community (PyTorch, PyG). Special thanks to the team division of responsibilities (A–D) that made the ablation, drift and adversarial tracks possible.

ABSTRACT

We watch network traffic and learn what *normal* looks like, without ever training on attacks. When something behaves abnormally — a scanner with high fan-out, a DDoS victim with many-to-one collapse, an exfiltrating host — we raise an edge-level alert `ScoredAlert[src_ip, dst_ip]`, explain why (SHAP/ATT&CK), and keep working even when the attack family has never been seen (zero-day premise). Flows (85-column CICIDS2017 GeneratedLabelledFlows, latin-1) are drawn as directed host graphs per 60s and 300s window (nodes=hosts, edges=flows). A GraphSAGE autoencoder ($2 \times \text{SAGEConv}$, hidden 32, latent 8 = in_dim) trained only on benign Monday learns to reconstruct normality; `NodeScaler(log=True)` log1p-before-min-max fixes power-law squash ($P@100$ 0.250 $\rightarrow$ 0.413, Patator 0 $\rightarrow$ 0.618). Scores are percentile-calibrated against a 20% Monday holdout and fused by within-window rank noisy-or $1 - \prod(1 - p_i)$; edge scores use `rank_mean` (node+edge, AUC 0.712 $\rightarrow$ 0.789). Evaluated under **HELD-OUT**: train benign Monday only, test each of 7 families held-out one-by-one, mean $\pm$ std over 4 seeded GPU-deterministic runs (`set_seed()` + `CUBLAS_WORKSPACE_CONFIG=:4096:8`). Production recipe **GNN-logscale 60s+300s + revived 87-dim ctx M5a, rank noisy-or $\rightarrow$ 0.9996 ± 0.0001** mean ROC-AUC across 7 families (v2 19-dim: 0.9997 ± 0.0001), every attacker in top ~ 35 of thousands (recall@100=1.0), replicated on IDS2018 (top-11/32,935, 3/4 seeds) and CTU-13 Virut #1 in 4/4 seeds. Beats re-run baselines under identical conditions (PCA 0.9417, IF 0.9357, MLP-AE 0.9517, +4.8 pts). Evasion costs $20 \times$ time (slow scan) or 16 machines (distributed); camouflage fails. Drift is monitored on three streams; SHAP explains each alert. One-command artifact: `python detection/eval_mw_ablation_4seed.py --seeds 0 1 2 3 --epochs 60`.

Keywords: zero-day, NIDS, host graphs, GraphSAGE, autoencoder, HELD-OUT, calibration, noisy-or, LogScaler, SHAP, concept drift.

CONTENTS

Certificate	1
Declaration	2
Acknowledgements	3
Abstract	4
Contents	5
List of Tables	7
List of Figures	8
List of Abbreviations	9
1 INTRODUCTION	10
1.1 Introduction	10
1.2 Background	10
1.2.1 Why Graphs?	11
1.3 Problem Statement	11
1.4 Objectives	12
1.5 Scope	12
1.6 Methodology — Summary	12
1.7 Organisation of the Report	12
1.8 Expected Outcome	13
2 REVIEW OF LITERATURE	14
2.1 Evaluation protocol	14
2.2 Datasets and flow features	14
2.3 Reconstruction-based detection	15
2.4 Graph and relational detection	15
2.5 Drift, adversarial cost, and explanations	16
2.6 UEBA and deployment context	17
2.7 Summary and gap	17
Bibliography	18
3 METHODOLOGY	21
3.1 Introduction	21
3.2 Datasets and Preprocessing	21
3.2.1 Primary — CICIDS2017	21
3.3 Host-Graph Construction	21
3.3.1 Time Windows	21
3.3.2 Nodes, Edges, Direction	21
3.3.3 Node and Edge Features	22
3.3.4 HostScaler	22

3.4	Models	22
3.4.1	M5a — Revived Per-Flow AE (87-dim)	22
3.4.2	M5b — GraphSAGE Host AE	22
3.4.3	Determinism	22
3.5	Fusion and Scoring	22
3.5.1	Calibration	22
3.5.2	Fusion Rules	23
3.5.3	Alert Emission	23
3.6	Drift, Explainability, Adversarial	23
3.7	Evaluation Protocol (HELD-OUT) and Metrics	23
3.8	Tools and Environment	23
4	DESIGN AND MODELLING	24
4.1	Introduction	24
4.2	System Architecture — Three Pillars and Four Persons	24
4.3	Data Flow and Module Map	24
4.4	Modelling Details	25
4.4.1	GraphAutoencoder	25
4.4.2	Windows, Calibration, Fusion	25
4.4.3	Checkpoints and Thresholds	25
4.5	Alert Pipeline and Dashboard	25
4.6	Validation and Results Summary — Week-4 Freeze	25
4.7	Deployment Considerations	26
4.8	Appendices Mapping	26
5	RESULTS, DISCUSSION AND FUTURE WORK	27
5.1	Results — Headline (as of 2026-08-25, week-4 freeze)	27
5.2	Discussion	27
5.3	Limitations	27
5.4	Future Work — Weeks 4–12 Roadmap	27
	References	28
A	Schemas & Feature Catalogue	30
B	Report Cards (RC-01...RC-32)	31
C	Code Listings	32
D	Supplementary Tables	33

LIST OF TABLES

- 1.1 Background analogy — e-waste to network anomaly. 11
- 1.2 Team mapping (CLAUDE.md). 12
- 1.3 Expected headline — week-4 freeze (GPU, CUDA-deterministic, 4 seeds). . . . 13

- 2.1 Benchmark captures and graph usability 15
- 2.2 Graph systems and their standing 16

- 3.1 Dataset releases and graph capability. 21
- 3.2 Fusion rules evaluated (6 configs, 4 seeds). 23

- 4.1 System module map — stage, implementation and I/O. 24
- 4.2 Validation summary (GPU, CUDA-deterministic, 4 seeds, 2026-08-25). 25
- 4.3 Failure modes and mitigations. 26
- 4.4 Appendices mapping (code & supplementary in appendices per guideline). . . . 26

LIST OF FIGURES

LIST OF ABBREVIATIONS

NIDS	Network Intrusion Detection System
NADS	Network Anomaly Detection System
WEEE	Waste Electrical and Electronic Equipment
AE	Autoencoder
GNN	Graph Neural Network
SAGE	GraphSAGE
HELD-OUT	Held-Out-Family Evaluation Protocol
ROC-AUC	Receiver Operating Characteristic — Area Under Curve
P@100	Precision at 100
RC	Report Card (experiments/report_cards.md)

CHAPTER 1

INTRODUCTION

1.1 Introduction

Zero-day attacks — exploits for vulnerabilities unknown to vendor or defender — drive the most consequential breaches. The zero-day has been defined as “a vulnerability in software or hardware that is unknown to the vendor and for which no patch or signature exists at the time of exploitation” (NIST SP 800-150). An intrusion detection system has been defined as “a system that monitors network or host activity for signs of malicious behaviour or policy violation and produces reports to a management station” (Debar et al., 1999). According to NIST CSF and MITRE ATT&CK any host that processes network communications and deviates from learned normality would come under anomalous behaviour. Globally, NIDS/NADS are the most commonly used terms for network defence. However, technically, zero-day detection is only a subset of anomaly detection applied to network and host behaviour.

Signature-based NIDS cannot detect zero-days by definition; they match only known patterns. This project learns *normal* network behaviour unsupervised and flags deviations, explains why, monitors drift when “normal” itself shifts, and measures what evasion would cost an attacker. Three pillars are planned: network flows (Pillar 1, Semester 7), identity/UEBA (Pillar 2), and host syscalls via eBPF (Pillar 3, weeks 4–12). This report covers Semester 7 — Pillar 1 at production depth plus scaffolding for Pillars 2/3.

1.2 Background

Enterprise networks generate millions of bidirectional flows. Each flow is one row: who talked to whom, which port, which protocol, how many bytes and packets, how long, and flag statistics — 80+ per-flow features per CICFlowMeter (Arash Habibi Lashkari, CIC). CICIDS2017 exists in two releases: the 79-column `MachineLearningCSV` (no IP columns) and the 85-column `GeneratedLabelledFlows` (has Flow ID, Source IP, Destination IP, Protocol, Timestamp — required for graphs, latin-1 encoded). Only the latter can build host graphs.

A scanner’s single flows look benign — a TCP SYN to port 22 appears normal. Its *fan-out* across a 60-second window does not: one host contacting 200 distinct peers in 60 seconds is a rate that the graph makes visible. Similarly, a DDoS victim shows many-to-one collapse, and infiltration shows a single internal host quietly exfiltrating to an external peer with unusual byte-entropy. Relational features — degree, in/out ratio, port entropy, unique peer count, flag ratios — capture these collective patterns that per-flow views discard.

The field has expanded rapidly. As per MITRE, annual new CVEs exceeded 20,000 during 2023–24. VERIZON DBIR estimates global cyber-crime cost at 8–10 Trillion USD per annum. The e-waste analogy from the reference example translates: just as WEEE comprises waste equipment not fit for original use, anomalies comprise traffic not fit for the learned model of normality — valuable if recovered, hazardous if mishandled.

Table 1.1: Background analogy — e-waste to network anomaly.

Fact (E-waste analogue)	Fact (Network analogue)
Annual e-waste India 0.8 Mt (2012), global 30–50 Mt	Annual CVEs >20k, alerts per enterprise 10k–100k/day
EEE contains valuables + hazardous toxics	Traffic contains benign signal + 1–8 attackers among thousands
Crude recovery (open burning) releases dioxins	Crude evaluation (training on attacks) leaks labels
WEEE: Plastics 30%, Oxydes 30%, Copper 20%	CICIDS2017: Benign 70%, PortScan 5%, DDoS 8%, Patator 1%

Source: Umweltbundesamt (2004), MITRE, VERIZON DBIR.

1.2.1 Why Graphs?

Per-flow detectors treat each flow independently. Graph detectors treat a 60-second or 300-second slice as a picture: each machine is a dot, each conversation a directed line. Degree and neighbourhood structure then become first-class signals. GraphSAGE (Hamilton et al., 2017) is chosen over GCN (Kipf & Welling, 2017) because GCN’s symmetric normalisation washes out the degree signal we are detecting — a design decision defended in `detection/gnn_model.py`. Time-based windows (not fixed-count) are essential: “200 peers in 60 seconds” is a rate. Edge-level alerts (`src_ip`, `dst_ip`) are required because the frozen `ScoredAlert` schema needs both endpoints.

1.3 Problem Statement

Four problems motivate this work — each mirrors a gotcha the project has already encountered and measured.

1. **P1. Closed-world classifiers overstate zero-day performance.** When training and testing families overlap, a classifier reports 0.97+ AUC. Under held-out evaluation (train on benign Monday only, test on a family never seen), many methods collapse to 0.47 worse than random on WebAttacks.
2. **P2. Per-flow detectors are structurally blind.** A scanner’s individual flows are benign-looking; only the fan-out across a window reveals it. Per-flow AE (M5a) swings 0.42–0.98 while the host-graph model (M5b) never drops below 0.906.
3. **P3. Power-law scaling squashes small attackers.** Raw host counts are power-law: the busiest host maps to 1.0 and every other squashes near 0. Patator and WebAttacks were stuck at $P@100 = 0.000$ across 294 runs until `NodeScaler(log=True)` fixed it to 0.618 and 0.381.
4. **P4. Uncalibrated, single-scale scores fuse incorrectly.** Raw M5a error (0.038–0.122) versus a rank (0–1) meant `max` picked the relational score on 99.9% of alerts, so every alert before 2026-08-12 was M5b alone.

1.4 Objectives

1. Baseline-vs-graph ablation (M5a per-flow 87-dim vs M5b GraphSAGE 8/19-dim vs fusion M5c) — the individually attributable headline.
2. Multi-window calibrated fusion (60s+300s percentile + rank noisy-or $1 - \prod(1 - p)$).
3. Honest held-out protocol — benign Monday only, 7 families, 4-seed GPU-deterministic bands.
4. Drift-aware SHAP + ATT&CK and adversarial harness (slow 20×, distributed 16×).

1.5 Scope

Table 1.2: Team mapping (CLAUDE.md).

Member	Track	Owns
A — Saharsh	Data & Capture	FlowRecord, pcap_to_flows, schema_mapper
B — Deep	Detection Modeling	M5a revived, GNN-temporal, drift, ensembler
C — Aditya	Trust & Risk	SHAP, UEBA, ATT&CK, privacy pass
D — Avinash	Adversarial & Delivery	Harness, FastAPI + React dashboard

In scope (Sem 7): schema mapping, host-graph construction with health gates, v1/v2 features, LogScaler, GraphSAGE ensemble, multi-window rank fusion, edge-level alerts, 4-seed HELD-OUT, drift/SHAP scaffolding. Out of scope: full UEBA and eBPF host AE (weeks 4–12, Knowledge/). Dataset gate: CICIDS2017 GeneratedLabelledFlows (85 cols, has IPs, latin-1). Synthetic training_data/ excluded — graphs collapse.

1.6 Methodology — Summary

Flows → host graphs per time window (nodes=hosts, edges=directed, graph_builder.py)
 → GraphSAGE autoencoder trained benign-only (M5b, gnn_model.py, NodeScaler log1p)
 → multi-window rank noisy-or fusion (M5c) → edge-level ScoredAlert queue(alert_pipeline.py)
 with SHAP + drift (M6/M7). Evaluation: HELD-OUT, 7 families, 4 seeds.

1.7 Organisation of the Report

Chapter I Introduction; Chapter II Review of Literature (presentation 10–15 September); Chapter III Methodology; Chapter IV Design and Modelling; Chapter V Results, Discussion & Future Work; References; Appendices (code, schemas, RC cards). Mirrors sample 1.8 DOCUMENTATION structure.

1.8 Expected Outcome

Table 1.3: Expected headline — week-4 freeze (GPU, CUDA-deterministic, 4 seeds).

Family	AUC (v2 fused)	Attacker ranks
PortScan / DoS / DDoS	0.9999–1.0000	1–4
WebAttacks / Patator	1.0000	1–5
Infiltration	0.9997	2
Botnet	0.9987	5–34

Source: eval_mw_ablation_4seed.py –seeds 0 1 2 3; eval_feature_set_v2.py

Mean ROC-AUC 0.9996 ± 0.0001 (v1) / 0.9997 ± 0.0001 (v2) across 7 held-out families; every attacker in top ~ 35 of thousands on CICIDS2017, top-11/32,935 on IDS2018 (3/4 seeds), CTU-13 Virut #1 in 4/4 seeds; recall@100=1.0; evasion needs $20\times$ time or 16 machines.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Evaluation protocol

This chapter reviews 40 distinct studies: protocol 3, datasets 6, reconstruction 4, graph 10, drift/adversarial/explain 15, UEBA 2. No duplicate merges were required in this register (the P12/P33 team-Excel merge is already recorded under [37]).

[1] Closed world (2010) — Sommer and Paxson [1] argue that training and testing on the same capture with overlapping families reports memorisation, not detection. Controls adopted here follow them: benign-only training, held-out attack families, a fixed alert unit, and multi-seed bands.

[2] Anomaly survey (2009) — Chandola et al. [2] frame network intrusions as collective anomalies: one scanner flow looks benign, the fan-out across a window does not. This is the premise for host-window aggregation in Chapter 3.

[3] IDS taxonomy (1999) — Debar et al. [3] separate signature systems (miss unseen families by definition) from anomaly systems (model normality, pay in thresholds and false alarms). The project is anomaly-side throughout.

2.2 Datasets and flow features

[4] CICIDS2017 (2018) — Sharafaldin et al. [4] supply the primary benchmark: five days, benign background plus seven attack families, about 2.8M flows. Only the 85-column GeneratedLabelledFlows release (Flow ID, Source/Destination IP, Protocol, Timestamp, latin-1) can build host graphs; the 79-column MLCSV release has no IP columns and feeds per-flow models only.

[5] LID-DS 2021 (2022) — Grimmer et al. [5] provide the future Pillar-3 host eval set (CRITIS 2022, LNCS 13723). The official baseline is STIDE; no benign-only Isolation Forest SOTA exists on LID-DS and none is quoted.

[6] CERT r6.2 (2013) — Glasser and Lindauer [6] generate the CMU SEI insider-threat set (4000 users, 5 malicious; logon, device, http, email, file). It is the Pillar-2 UEBA reference, not NIDS data.

[7] CTU-13 (2014) — Garcia et al. [7] publish 13 botnet scenarios for external replication; infected hosts rank first in our CTU-13 runs.

[8] UNSW-NB15 (2015) — Moustafa and Slay [8] supply the hybrid synthetic-plus-real set (nine families) used as a second external check.

[9] CICFlowMeter (2017) — Lashkari [9] defines the flow vocabulary: 80+ per-flow statistics over packet lengths, inter-arrival times, flags, and segment sizes. Two preprocessing constraints dominate architecture choice: NaN-address rows (Thursday WebAttacks, about 63% unusable) must be dropped before host-set construction, and power-law byte/packet counts require log compression before scaling.

Table 2.1: Benchmark captures and graph usability

Capture	Scale	Graph-usable
CICIDS2017 [4]	≈2.8M flows, 7 families	85-col release only
CSE-CIC-IDS2018	≈16M flows	1 of 10 files
CTU-13 [7]	13 scenarios	Yes, with column map
UNSW-NB15 [8]	≈2.5M flows, 9 families	Yes

Source: dataset publications above; usability determined in Chapter 3.

2.3 Reconstruction-based detection

[10] Autoencoder anomaly (2014) — Sakurada and Yairi [10] establish bottleneck reconstruction error as the anomaly score. The per-flow baseline (M5a) and the host autoencoder (Pillar 3) both inherit this objective.

[11] Kitsune (2018) — Mirsky et al. [11] distribute capacity across per-cluster experts for online packet scoring, but still score flows rather than neighbourhoods.

[12] Isolation Forest (2008) — Liu et al. [12] and one-class SVM/PCA references are the shallow baselines beaten under identical conditions in RC-31 (best baseline 0.9517 vs ours 0.9996).

[13] VAE reconstruction probability (2015) — An and Cho [13] (SNU technical report SNUDM-TR-2015-03, never a venue paper) score by latent-distribution deviation; retained as the probabilistic baseline, cited without venue.

2.4 Graph and relational detection

[14] GraphSAGE (2017) — Hamilton et al. [14] use mean neighbourhood aggregation, which preserves the degree signal. GCN [15], marker [15], uses symmetric normalisation and smooths it away.

[15] GCN (2017) — Kipf and Welling [15] supply the contrast backbone: for degree-defined behaviours (one-to-many scan, many-to-one flood) symmetric normalisation attenuates exactly the attack signal.

[16] PIKACHU (2022) — Paudel and Huang [16] learn temporal-walk dynamic embeddings, evaluated on provenance captures (DARPA OpTC/LANL APT replays), not held-out families. The exact recall figure is withheld: the NOMS’22 paper is paywalled and no abstract, repo, or secondary source pins it.

[17] E-GraphSAGE (2022) — Lo et al. [17] share the GraphSAGE backbone with edge features but train supervised (NOMS’22, not TDSC as previously cited): unseen families fall outside decision regions by construction.

[18] Anomal-E (2022) — Caville et al. [18] learn edge embeddings self-supervised and are closest in framing, leaving alert granularity open.

[19] Pre-training leakage (2024) — Gu et al. [19] show target encoding of categorical fields leaks labels into supposedly unsupervised pipelines; in-context dense representations avoid it. Our pipeline pins vocabularies from benign-train only.

[20] R+R repro (2025) — Wang et al. [20] (ACSAC 2025; camera-ready title “R+R: From Claims to Crashes”, cited via arXiv:2503.20281) reproduce five GIDS and find replication deviates substantially with metric, preprocessing, and hyperparameters (Fig. 4), collapse on

CICIDS2017/enterprise with high false positives, and full LANL evasion with two adversarial edges. This is the caution behind our seed bands and preprocessing pins.

[21] **MAGIC (2024)** — Jia et al. [21] (33rd USENIX Security) detect APTs with a masked graph autoencoder plus benign-only K-D-tree kNN outlier: the closest host-side analogue of M5b.

[22] **Unicorn (2020)** — Han et al. [22] encode longitudinal provenance in fixed-size sketches: the drift/scalability precedent for long-running host monitoring.

[23] **ThreaTrace (2022)** — Wang et al. [23] classify node roles with GraphSAGE and score predicted-vs-actual type: the direct host-side node analogue. No SOTA-beaten count is quoted (sources disagree between three and seven).

Table 2.2: Graph systems and their standing

System	Backbone	Standing
GraphSAGE [14]	Mean aggregation	Preserves degree
GCN [15]	Symmetric norm.	Washes out degree
PIKACHU [16]	Dynamic walks	Provenance eval only
E-GraphSAGE [17]	Edge SAGE, supervised	No zero-day coverage
Anomal-E [18]	Self-supervised GNN	Granularity open
R+R [20]	Repro audit	Cautionary baseline

Source: publications above; verdicts verified September 2026.

2.5 Drift, adversarial cost, and explanations

[24] **Drift survey (2014)** — Gama et al. [24] distinguish supervised detectors that watch labels from unsupervised ones that watch score streams; only the latter applies to zero-day deployment, motivating the M6 monitor.

[25] **Real attackers (2023)** — Apruzzese et al. [25] (IEEE SaTML, not USENIX) show realistic attackers do not compute gradients: the harness threat model is problem-space rewrites, not feature noise.

[26] **Robustness benchmark (2023)** — Vitorino et al. [26] rank robustness across models and captures with fixed threats instead of single headlines.

[27] **Galli defense (2025)** — Galli et al. [27] (23rd IEEE NCA, Lisbon) give the structural adversarial-training defense for GNN-NIDS tested on CTU-13/TON-IoT with E-GraphSAGE: clean warm-up, then hardening on benign flows with low-degree endpoints (Menti-botnet mean detection rate across C2xB steps 0.027 to 0.926). This is our hardening template.

[28] **TANTRA (2022)** — Sharon et al. [28] reshape only inter-packet timing (average success 99.99% over 8 attacks by 3 NIDS): the harness timing-attack class against flow time features.

[29] **A2PM (2022)** — Vitorino et al. [29] (official spelling “Adaptative”) constrain adversarial patterns to valid, coherent network behaviour: the realism filter that rejects gradient noise breaking port, protocol, or attack function.

[30] **MARS (2024)** — Huang et al. [30] certify supervised flow classifiers with adaptive randomized smoothing. Certified, but for classifiers, not autoencoders: no transfer is claimed.

[31] **SSF replay (2025)** — Zhang et al. [31] (INFOCOM 2025) detect drift, select drift-representative samples, strategically forget, and fine-tune: the M6 response blueprint (refresh memory, never blind retrain).

[32] **MMD continual (2026)** — Xu et al. [32] pair feature-space MMD drift detection with gradient-matching coresets. No dependency on [31] is claimed.

[33] **ReCDA (2024)** — Yang et al. [33] (KDD 2024; journal version TDSC 22(6), 2025) adapt representations to drift: the comparator for continual regimes.

[34] **SHAP (2017)** — Lundberg and Lee [34] (Proc. NIPS, Long Beach) give additive per-alert attributions used for triage in M7.

[35] **LIME/SHAP on MLP (2024)** — Gaspar et al. [35] (IEEE Access, vol. 12) compare post-hoc explainers on an MLP-IDS over generated ADFA-LD IoT data.

[36] **Fooling SHAP (2020)** — Slack et al. [36] (AIES, not FAT*) fool LIME fully and SHAP 84–100% under single-feature scaffolding (67–71% with two features) via OOD scaffolding: the explainer is robustness-checked, never ground truth.

[37] **XAI-IDR (2026)** — Panchal et al. [37] gate automated containment on prediction confidence plus explanation stability, escalating the rest: the safe-automation policy.

[38] **Human-centred NLE (2026)** — Padin-Torrente et al. [38] render anomaly explanations in natural language for analyst consumption.

2.6 UEBA and deployment context

[39] **UEBA autoencoder (2025)** — Fuentes et al. [39] detect threats with deep autoencoders over behavioural features: the Pillar-2 modelling precedent.

[40] **UBA review (2026)** — Akampurira et al. [40] (F1000Research, v1, lightly reviewed: low-weight like the tail-risk study) systematically review deep UBA architectures with ISO/IEC 27001 alignment.

Industry systems (Darktrace, Vectra, ExtraHop, Cisco ETA, Defender XDR, QRadar to XSIAM, SANS 2024 survey) inform deployment lessons only: per-environment calibration, metadata-without-decryption evaluation, false-positive burden (64% major challenge), and migration planning. They carry no archival numbers and enter no reference list.

2.7 Summary and gap

Unsupervised reconstruction handles unseen families, graph context captures collective behaviours invisible per-flow, and calibration plus held-out evaluation decide whether numbers transfer. The gap addressed here is narrow: unsupervised host-graph reconstruction with benign-only training, edge-level alerts with node/edge numbers quoted separately, calibrated multi-window fusion with seed bands, held-out-family protocol on CICIDS2017 with IDS2018/CTU-13 replication, a score-stream drift monitor with response, a cost-reporting red-team harness, and robustness-checked SHAP. No cited paper combines all seven.

Chapter References

BIBLIOGRAPHY

- [1] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.
- [2] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, Art. no. 15, pp. 1–58, 2009.
- [3] H. Debar, M. Dacier, and A. Wespi, “Towards a taxonomy of intrusion-detection systems,” *Comput. Networks*, vol. 31, no. 8, pp. 805–822, 1999.
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.
- [5] M. Grimmer *et al.*, “Dataset report: LID-DS 2021,” in *Critical Information Infrastructures Security (CRITIS 2022)*, LNCS 13723, pp. 63–73, Springer, 2023.
- [6] J. Glasser and B. Lindauer, “Bridging the gap: A pragmatic approach to generating insider threat data,” in *Proc. IEEE Symp. Security and Privacy Workshops (SPW)*, San Francisco, CA, 2013, pp. 98–104.
- [7] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, “An empirical comparison of botnet detection methods,” *Comput. Secur.*, vol. 45, pp. 100–123, 2014.
- [8] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. Military Commun. Inf. Syst. Conf. (MilCIS)*, Canberra, Australia, 2015.
- [9] A. H. Lashkari, CICFlowMeter (formerly ISCXFlowMeter), Canadian Institute for Cybersecurity, 2017. Available: <https://github.com/ahlashkari/CICFlowMeter>
- [10] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proc. MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, 2014, pp. 4–11.
- [11] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. NDSS*, 2018.
- [12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.
- [13] J. An and S. Cho, “Variational autoencoder based anomaly detection using reconstruction probability,” SNU Data Mining Center, Tech. Rep. SNUDM-TR-2015-03, Dec. 2015.
- [14] W. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. NeurIPS*, 2017.
- [15] T. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.

- [16] R. Paudel and H. Huang, “PIKACHU: Temporal walk based dynamic graph embedding for network anomaly detection,” in *Proc. IEEE/IFIP NOMS*, 2022.
- [17] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, “E-GraphSAGE: A graph neural network based intrusion detection system for IoT,” in *Proc. IEEE/IFIP NOMS*, Budapest, Hungary, 2022, pp. 1–9.
- [18] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, “Anomal-E: A self-supervised network intrusion detection system based on graph neural networks,” *Knowl.-Based Syst.*, vol. 258, p. 110030, 2022.
- [19] Z. Gu, D. T. Lopez, L. Alrahis, and O. Sinanoglu, “Always be pre-training: Representation learning for network intrusion detection with GNNs,” in *Proc. ISQED*, 2024.
- [20] C. Wang, P. Zheng, J. Gui, C. Hua, and W. U. Hassan, “R+R: From claims to crashes: A systematic re-evaluation of graph-based network intrusion detection systems,” in *Proc. ACSAC*, Honolulu, HI, USA, 2025. arXiv:2503.20281.
- [21] Z. Jia *et al.*, “MAGIC: Detecting advanced persistent threats via masked graph representation learning,” in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, 2024, pp. 5197–5214.
- [22] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “UNICORN: Runtime provenance-based detector for APTs,” in *Proc. NDSS*, 2020.
- [23] S. Wang *et al.*, “THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3972–3987, 2022.
- [24] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, Art. 44, 2014.
- [25] G. Apruzzese *et al.*, “Real attackers don’t compute gradients: Bridging the gap between adversarial ML research and practice,” in *Proc. 1st IEEE Conf. Secure and Trustworthy Machine Learning (SaTML)*, 2023.
- [26] J. Vitorino, M. Silva, E. Maia, and I. Praca, “An adversarial robustness benchmark for enterprise network intrusion detection,” in *Proc. 16th Int. Symp. Foundations and Practice of Security (FPS)*, Bordeaux, France, 2023, LNCS 14551, pp. 3–17, Springer, 2023.
- [27] D. Galli, A. Venturi, D. Stabili, M. Andreolini, and M. Marchetti, “Defending network intrusion detection systems based on graph neural networks against structural adversarial attacks,” in *Proc. 23rd IEEE Int. Symp. Netw. Comput. and Appl. (NCA)*, Lisbon, Portugal, 2025, pp. 219–228.
- [28] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, “TANTRA: Timing-based adversarial network traffic reshaping attack,” *IEEE Trans. Inf. Forensics Security*, vol. 17, 2022.
- [29] J. Vitorino, N. Oliveira, and I. Praca, “Adaptative perturbation patterns: Realistic adversarial learning for robust intrusion detection,” *Future Internet*, vol. 14, no. 4, p. 108, 2022.

- [30] M. Huang, Y. Lin, X. Chen, and E. Bertino, “MARS: Robustness certification for deep network intrusion detectors via multi-order adaptive randomized smoothing,” in *Proc. IEEE TrustCom*, Sanya, China, 2024, pp. 767–774.
- [31] X. Zhang *et al.*, “Continual learning with strategic selection and forgetting for network intrusion detection,” in *Proc. IEEE INFOCOM*, 2025.
- [32] B. Xu *et al.*, “A coordinated continual intrusion detection approach with feature-space MMD drift detection and gradient-matching coresets,” *Mathematics*, vol. 14, no. 14, p. 2595, 2026.
- [33] S. Yang *et al.*, “ReCDA: Concept drift adaptation with representation enhancement for network intrusion detection,” in *Proc. 30th ACM SIGKDD*, Barcelona, 2024, pp. 3818–3828.
- [34] S. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, 2017.
- [35] D. Gaspar, P. Silva, and C. Silva, “Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron,” *IEEE Access*, vol. 12, pp. 30164–30175, 2024.
- [36] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, “Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods,” in *Proc. AAAI/ACM AIES*, 2020, pp. 180–186.
- [37] R. V. Panchal, R. Snehkunj, and V. V. Panchal, “Explainable artificial intelligence (XAI) for intelligent intrusion detection systems and threat response automation (XAI-IDR),” *Int. J. Computer Applications*, vol. 187, no. 74, pp. 51–55, Jan. 2026.
- [38] H. Padin-Torrente, V. Carneiro-Diaz, and I. Ortega-Fernandez, “Toward human-centered explainability: Natural language explanations for anomaly detection,” *Information Systems Frontiers*, 2026.
- [39] J. Fuentes *et al.*, “Cybersecurity threat detection based on a UEBA framework using deep autoencoders,” *AIMS Mathematics*, vol. 10, no. 10, pp. 23496–23517, 2025.
- [40] P. Akampurira *et al.*, “A deep learning-based user behavior analytics model for proactive cyber threat detection and risk management,” *F1000Research*, vol. 15, p. 674, 2026.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter details data, graph construction, HostScaler, GraphSAGE autoencoder, calibration, multi-window fusion, HELD-OUT protocol, metrics, reproducibility and tools. Every choice is traceable to a gotcha or report card. Time-based windows, directed edges, SAGE over GCN, log1p before scaling, and edge-level alerts are defended decisions.

3.2 Datasets and Preprocessing

3.2.1 Primary — CICIDS2017

Monday 2017-07-03 benign only training; Tuesday–Friday 7 families held-out. Two releases only latter builds graphs (GOTCHA #3). All GeneratedLabelledFlows are latin-1; Thursday WebAttacks 63% junk handled by `drop_unusable_rows()`. Naming drift handled by `schema_mapper`; never `dropna(axis=1)` per-file — pin via `pin_canonical`. Feature catalogue: 87-dim flows (76 CIC +11 ctx) and 8/19-dim hosts (0–7 stable).

Table 3.1: Dataset releases and graph capability.

Release	Cols	Has IPs?	Use
MachineLearningCSV	79	No	Per-flow M5a only
GeneratedLabelledFlows	85 (latin-1)	Yes	Host graphs (production gate)
IDS2018 (10 CSVs)	varies	1/10 only	Thursday only graphable

Synthetic `dataset_10k_normal.csv` (508 clients → 2 services, collapsed) and `live_capture.csv` (12,503 services) cannot form graphs — `graph_health()` rejects. External: IDS2018 only Tuesday, CTU-13 Virut/Rbot/Neris, planned UNSW-NB15.

3.3 Host-Graph Construction

3.3.1 Time Windows

Time-based 60s and 300s (rate vs volume trade; 60s maximises P@100; we fuse both). 60s → 1800s AUC 0.917 → 0.983 but P@100 0.244 → 0.093 (RC-02).

3.3.2 Nodes, Edges, Direction

Nodes=hosts, Edges=directed, no self-loops, no collapse. SAGE over GCN because GCN symmetric normalisation washes out degree.

3.3.3 Node and Edge Features

$$\begin{aligned}\text{out_degree}(v) &= |\{u : (v \rightarrow u) \in E\}| \\ \text{port_entropy}(v) &= - \sum p(\text{port}) \log p(\text{port}) \\ \text{bytes_sent}(v) &= \sum \text{bytes}(f)\end{aligned}$$

Edge_attr (4-dim) scored via rank_mean lifts edge AUC 0.712 $\rightarrow$ 0.789 (+P@100 0.207 $\rightarrow$ 0.349) — only change improving both (GOTCHA #19).

3.3.4 HostScaler

$x_{\log} = \log(1+x)$, $x_{scaled} = (x_{\log} - \min_{\log}) / (\max_{\log} - \min_{\log})$ fit on benign only. NodeScaler(log=True) default since 2026-08-12; mean P@100 0.250 $\rightarrow$ 0.413, Patator 0 $\rightarrow$ 0.618.

3.4 Models

3.4.1 M5a — Revived Per-Flow AE (87-dim)

Legacy 76-dim shim threshold 0.5 uncalibrated (benign max $\sim$ 0.278). Revived 87-dim (76+11 ctx) m5a_revived_ctx.pt — not in production defaults but headline uses it with fusion.

3.4.2 M5b — GraphSAGE Host AE

$2 \times \text{SAGEConv } N \times 8/19 \rightarrow N \times 32 \rightarrow N \times 8$ latent (=in_dim, no bottleneck — capacity via features, latent 2 $\rightarrow$ 12 moves mean 0.003). Decoder: $\hat{x}_v = \text{MLP}(z_v)$, $\hat{a}_{uv} = \text{MLP}([z_u; z_v])$, loss $\text{MSE}(x, \hat{x}) + \text{MSE}(a, \hat{a})$, score $\|x - \hat{x}\|^2 + \lambda \|a - \hat{a}\|^2$.

3.4.3 Determinism

set_seed(): torch.manual_seed, cuda.manual_seed_all, cudnn.deterministic=True, benchmark=False, CUBLAS_WORKSPACE_CONFIG=:4096:8. Same seed CPU 0.5048 vs GPU 0.9948 (GOTCHA #24); single-seed <6 pts is noise.

3.5 Fusion and Scoring

3.5.1 Calibration

Per-window percentile against 20% Monday holdout (80/20). Ensemble scores are percentiles not raw errors (GOTCHA #18). Uncalibrated until 2026-08-12 was M5b alone 99.9% (GOTCHA #21).

3.5.2 Fusion Rules

Table 3.2: Fusion rules evaluated (6 configs, 4 seeds).

Rule	Formula	Property
mean	$(p60 + p300)/2$	Baseline
max	$\max(p60, p300, pM5a)$	Lets saturated M5a overwrite
rank_mean	$\text{rank}(\text{pop}) \rightarrow \text{mean}$	Window-local, lifts $0.712 \rightarrow 0.789$
fused_rank_max	rank positions	+0.0398 AUC but batch-only
noisyor (prod)	$1 - \prod(1 - p)$	0.9996 ± 0.0001 best

Production: within-window rank noisy-or over 60s+300s (plus M5a ctx where enabled), edge rank_mean. `score_window(feature_columns=None)=M5b-only 0.8322` edge AUC most consistent.

3.5.3 Alert Emission

`score_window() → ScoredAlert[src_ip, dst_ip, score, rank, model_source]` sorted by fused rank. `feature_set="v2"` without 19-dim checkpoint refuses loudly. We report node AUC for modelling, edge AUC for operational (gap 22 pts, GOTCHA #16).

3.6 Drift, Explainability, Adversarial

M6 `drift_monitor.py` on 3 streams via `init_drift_monitors()` (queue noise is drift RC-28). **M7** SHAP + ATT&CK (never unpack positionally). **Harness** 4 techniques: port-hiding no, slow 20×, distributed 16, camouflage fail (RC-14).

3.7 Evaluation Protocol (HELD-OUT) and Metrics

HELD-OUT: train Monday benign only, test each family held-out one-by-one, mean±std over seeds 0–3 (`eval_mw_ablation_4seed.py`). Metrics: ROC-AUC primary, attacker rank & recall@100 operational (top ~35, recall 1.0), P@100 diagnostic capped at bad/100. Node vs edge quoting gap stated. Closed negatives: shipped-M5a, temporal, LODO, k=5, small-window, latent width.

3.8 Tools and Environment

Python 3.11, PyTorch 2.5.1+cu121, PyG; `venv` per-machine never portable; paths via `Path(__file__).resolve()`. `data/` gitignored (CIC gates), *.pt tracked. One-command: `python detection/eval_mw_ablation.py --seeds 0 1 2 3 --epochs 60` (GPU deterministic).

CHAPTER 4

DESIGN AND MODELLING

4.1 Introduction

Architecture, component design, data flow and modelling choices implementing Chapter 3. Decisions defended: AE never classifier, SAGE over GCN, nodes=hosts/edges=directed, time-based windows, edge-level alerts, M5a lifted to host-window. Three pillars: network (P1 sem 7), UEBA (P2), eBPF syscalls (P3 weeks 4–12). Headline is baseline-vs-GNN ablation (B).

4.2 System Architecture — Three Pillars and Four Persons

Pillar 1 Network: `capture/schema_mapper` → `pcap.to.flows` → `graph_builder` → `gnn_model` (LogScaler) → `ensampler` (60s+300s rank noisy-or) → `alert_pipeline` → `dashboard` (FastAPI+React). Pillar 2 UEBA: identity logs → risk model. Pillar 3 Host: `ebpf_syscall_watcher` → host AE. Cross-cutting: M6 drift_monitor, M7 shap_explainer, harness D. Named method HOSTFUSE + HELD-OUT, one-command artifact.

4.3 Data Flow and Module Map

Table 4.1: System module map — stage, implementation and I/O.

S.No.	Stage	Module	Input → Output
1	Ingest — schema	<code>capture/schema_mapper</code>	CSV (79/85/IDS2018) → canonical FlowRecord
2	Ingest — capture	<code>pcap.to.flows</code> / <code>ebpf_syscall_watcher</code>	pcap/eBPF → flows / syscalls
3	Build — graph	<code>graph_builder.py</code>	flows → List[Data] x=[N,8/19]
4	Scale	<code>gnn_model:NodeScaler</code>	raw-log → min-max
5	Train — baseline	<code>train_m5a_revived.py</code>	87-dim → <code>m5a_revived.ctx.pt</code>
6	Train — GNN	<code>gnn_model:GraphAutoencoder</code>	benign graphs → <code>gnn_autoencoder.v1*.pt</code>
7	Score — fuse	<code>ensampler.py</code>	60s+300s errors → percentiles → noisy-or
8	Serve — alerts	<code>alert_pipeline.py</code>	window → ScoredAlert[src,dst,score,rank]
9	Evaluate — lab	<code>eval_mw_ablation_4s</code>	held-out loop → mean±std
10	Monitor	<code>drift_monitor.py</code>	3 streams → drift flag
11	Explain	<code>shap_explainer.py</code>	embedding → SHAP + ATT&CK
12	Adversarial	<code>harness/graph_techniques</code>	4 attacks → cost

Source: *detection/README.md, experiments/report_cards.md*

Health gate: `graph_health()` rejects collapsed (508→2 services) and degenerate (12,503 services) before training.

4.4 Modelling Details

4.4.1 GraphAutoencoder

$2 \times \text{SAGEConv } N \times 8/19 \rightarrow N \times 32 \rightarrow N \times 8$ latent (=in_dim), ReLU, decoder MLPs, loss $\text{MSE}(\text{node}) + \text{MSE}(\text{edge})$, score via `edge_score rank_mean`. Latent width null (0.003), add features not width. v1 8 dims vs v2 19 dims (adds port/peer entropy, flag ratios, IAT); guard refuses v2 without checkpoint; 0–7 stable.

4.4.2 Windows, Calibration, Fusion

60s+300s fused after percentile calibration; checkpoints must ship together (percentiles not raw). Fusion: $p60 = \text{percentile}(\text{err60})$, $\text{fused} = 1 - (1 - \text{rank}(p60))(1 - \text{rank}(p300))(1 - \text{rank}(pM5a))$ (M5b-only omit pM5a $\rightarrow$ 0.8322 edge AUC).

4.4.3 Checkpoints and Thresholds

Ensemble scores are percentiles; `DEFAULT_THRESHOLD` 0.5 uncalibrated (benign max ~ 0.278). `model_source` records single vs ensemble.

4.5 Alert Pipeline and Dashboard

`ScoredAlert` $\rightarrow$ `alert_pipeline.score_window()` $\rightarrow$ FastAPI GET `/alerts` + Web-Socket $\rightarrow$ React dashboard (filters, SHAP drawer, drift banner). Run: `run_detector.bat` or `powershell -ExecutionPolicy Bypass -File .\run_detector.ps1` or VS Code `stub_detector.py` (deprecated shim).

4.6 Validation and Results Summary — Week-4 Freeze

Table 4.2: Validation summary (GPU, CUDA-deterministic, 4 seeds, 2026-08-25).

Claim	Number	Source
CICIDS2017 v1	0.9996 ± 0.0001	<code>mw_ablation_4seed.json</code>
CICIDS2017 v2	0.9997 ± 0.0001	<code>feature_set_v2_results.json</code>
vs baselines	PCA 0.9417 / IF 0.9357 / MLP 0.9517	<code>baselines_4seed.json</code>
IDS2018 external	top-11/32,935 in 3/4 seeds	<code>external_ids2018_multiseed</code>
CTU-13 Virut	#1 in 4/4 seeds	<code>external_ctu13_multiseed</code>
CTU-13 Rbot	#1 in 3/4 seeds	same

Per-family v2: PortScan/DoS/DDoS 0.9999–1.0000 (1–4), Web/Patator 1.0000 (1–5), Infiltration 0.9997 (2), Botnet 0.9987 (5–34). Recall@100 = 1.0; P@100 not headlined. Closed negatives: shipped-M5a poison, temporal/LSTM neutral, LODO, k=5, small-window, latent width.

4.7 Deployment Considerations

Table 4.3: Failure modes and mitigations.

Component	Failure	Mitigation
CSV ingest	latin-1 vs UTF-8, NaN IPs	read_flows()+drop_unusable_rows()
Scaling	power-law squash	NodeScaler(log=True)
Reproducibility	unseeded/GPU nondeterminism	set_seed()+CUBLAS :4096:8
Portability	hard-coded D:\Test OD	Path(_file_).resolve().parent
Portability	venv absolute paths	Per-machine venv

Known weaknesses first: noisier batch-only, calibration optimism (20% holdout), device sensitivity, both checkpoints required, near-ceiling dataset.

4.8 Appendices Mapping

Table 4.4: Appendices mapping (code & supplementary in appendices per guideline).

Appendix	Content	Pointer
A — Schemas	feature_vector.json v3.0 (87-dim), ScoredAlert	detection/training_features/README.md
B — Evidence	report_cards.md RC-01...RC-32	Detail behind CHANGELOG
C — Code	graph_builder, gnn_model, ensembler, alert_pipeline	detection/ prod
D — Supplementary	baselines, externals, ablation bands	PAPER_OUTLINE

CHAPTER 5

RESULTS, DISCUSSION AND FUTURE WORK

5.1 Results — Headline (as of 2026-08-25, week-4 freeze)

Production recipe GNN-logscale 60s+300s + revived 87-dim ctx M5a, within-window rank noisy-or → **mean ROC-AUC 0.9996 ± 0.0001** across 7 held-out families (`eval_mw_ablation_4seed.py --seeds 0 1 2 3`), beats pure rank_mean 0.9990 ± 0.0003 in all seeds. With v2 (19 dims): **0.9997 ± 0.0001** (`eval_feature_set_v2.py`). Every attacker in top ~ 35 of thousands; top-11/32,935 on IDS2018 in 3/4 seeds (worst top-35, percentile 0.0002); CTU-13 Virut #1 in 4/4 seeds, Rbot #1 in 3/4.

Baselines beaten under identical conditions (RC-31): PCA 0.9417, IF 0.9357, MLP-AE 0.9517 vs 0.9996 (+4.8 pts). Closed negatives confirmed: shipped-M5a, temporal/LSTM, LODO, k=5.

5.2 Discussion

Operational claim to quote is attacker rank/recall, not P@100 (structurally capped at bad/100). Node AUC describes model, edge AUC describes alert queue (gap 22 pts). Calibration optimism (20% Monday holdout) and device sensitivity (CPU vs GPU WebAttacks 0.50→0.99) are stated before an examiner does. Graph context captures collective behaviours invisible per-flow; LogScaler is the single biggest win; SAGE preserves degree.

5.3 Limitations

Noisy-or batch-only (needs population to rank; single-alert uses rank_mean). Both checkpoints must ship together. Single-lab benign day. Drift monitor built but not yet auto-governing threshold. Feature v2 default flip pending sign-off.

5.4 Future Work — Weeks 4–12 Roadmap

Pillar 3 host-syscall autoencoder via eBPF (`capture/ebpf_syscall_watcher.py`), AE-vs-HMM ablation, three-way score fusion (network + UEBA + host). Team work — B supports. Third dataset replication (UNSW-NB15 with pseudo-timestamps), drift-governed threshold, and paper packaging when results freeze (HOSTFUSE + HELD-OUT, one-command artifact, protocol paper outline per `docs/PROJECT_GUIDE.md` §6).

REFERENCES

1. Debar, H. et al. Towards a taxonomy of IDS. *Comput. Networks*, 1999.
2. Axelsson, S. IDS: A survey and taxonomy. Chalmers, 2000.
3. Chandola, V. et al. Anomaly detection: A survey. *ACM CSUR*, 2009.
4. Garcia-Teodoro, P. et al. Anomaly-based NIDS. *IEEE Commun. Surv. Tuts.*, 2009.
5. Bhuyan, M. et al. Network anomaly detection. *IEEE Commun. Surv. Tuts.*, 2014.
6. Sommer, R. & Paxson, V. Outside the closed world. *IEEE S&P*, 2010.
7. Patcha, A. & Park, J. An overview of anomaly detection. *Comput. Networks*, 2007.
8. Hodge, V. & Austin, J. A survey of outlier detection. *Artif. Intell. Rev.*, 2004.
9. Lashkari, A.H. et al. *CICFlowMeter*. CIC, 2017.
10. Moore, A. et al. Discriminators for flow classification. 2005.
11. Williams, N. et al. Comparison of five ML algorithms. 2006.
12. Sharafaldin, I. et al. *CICIDS2017*. CIC, 2018.
13. Ring, M. et al. Survey of NIDS datasets. 2019.
14. Heng, M. et al. Dataset realism. 2024.
15. Durieux, V. & Gevenois, P. Bibliometric indicators. 2010.
16. Franceschini, F. h-index. 2010.
17. Zitt, M. & Bassecoulard, E. Challenges for indicators. 2008.
18. Biglia & Butler. Bibliometric analysis. 2005.
19. Sakurada, M. & Yairi, T. Anomaly detection using autoencoders. *MLSDA*, 2014.
20. An, J. & Cho, S. Variational autoencoder anomaly detection. 2015.
21. Liu, F. et al. Isolation Forest. *ICDM*, 2008.
22. Scholkopf, B. et al. Support vector novelty. *NeurIPS*, 1999.
23. Merton, R. 1963. Publication collaboration.
24. Manten. 1968. Earth Science authorship.
25. Zuckerman. 1968. Nobel laureates.
26. Meadows, A. 1974. Communication in science.
27. Klaic. 1990. Chemists of Rugjer Boskovic.
28. Vimala & Pulla Reddy. 1996. Zoology theses.
29. Gupta & Karisiddappa. 1998. Population genetics.
30. Mahapatra & Bhagavan Doss. 2000. Geology.
31. Seglen & Aksnes. 2000. Norwegian microbiology.
32. Moed, H. 2000. Biotechnology departments.
33. Arunachalam & Doss. 2000. Asian collaboration.
34. Kannappanavar & Vijayakumar. 2001. Authorship trend.
35. Farahat, H. 2002. Egyptian journals.
36. Suresh Kumar & Garg. 2005. China vs India CS.
37. Calero, C. et al. 2006. Bibliometric mapping.
38. Park, T. 2008. Library science authorship.

39. Tijssen & Van Leeuwen. 2001. Scientometric evaluation.
40. Thomaidis, N. et al. 2003. Balkan chemistry.
41. Leydesdorff, L. 2004. Research evaluation.
42. Surulinathi et al. 2007. Knowledge management India.
43. Tseng, Y. et al. 2009. Trend analysis.
44. Annibaldi et al. 2010. Italian analytical chemistry.
45. Padma, P. 2010. Madurai Kamaraj thesis.
46. Garfield, E. 1983. World research 1973.
47. Mehrotra & Lancaster. 1984. Indian science.
48. Raghuram & Madhavi. 1996. Decline in Indian science. Nature.
49. Raina, D. et al. 1995. Indian research 1800-1950.
50. Bandyopadhyay, A.K. 2001. Indian research 1950-1990.
51. Arunachalam et al. 1998. 42,000 Indian papers.
52. Arunachalam. 2000. Is science in India on decline.
53. Arunachalam & Gunasekaran. 2002. Tuberculosis.
54. Kademani, B.S. et al. 2006. Thorium India.
55. Kademani et al. 2007. Indian S&T 1990-2004.
56. UNEP E-waste Manual; NIST CSF; ENISA.
57. Hamilton, W. et al. GraphSAGE. NeurIPS, 2017.
58. Kipf, T. & Welling, M. GCN. ICLR, 2017.
59. Mirsky, Y. et al. Kitsune. NDSS, 2018.
60. Alsham et al. PIKACHU. 2022.
61. Lo, W. et al. E-GraphSAGE. TDSC, 2022.
62. Cavanaugh et al. Anomal-E. 2022.
63. Gama, J. et al. Concept drift. ACM CSUR, 2014.
64. Apruzzese et al. Real attackers don't compute gradients. USENIX, 2023.
65. Lundberg & Lee. SHAP. NeurIPS, 2017.
66. MITRE CVE; VERIZON DBIR; CIC IDS2018; UNSW-NB15; CTU-13.

CHAPTER A

SCHEMAS & FEATURE CATALOGUE

schemas/feature_vector.json v3.0: 76 $\rightarrow$ 87 dims (flow + ctx window). ScoredAlert fields: src_ip, dst_ip, score, rank, model_source. HostScaler params: log flag, min/max per dim.

CHAPTER B

REPORT CARDS (RC-01...RC-32)

See `experiments/report_cards.md` — one card per experiment with numbers, bands and caveats.
RC-26 is reference ablation (`eval_mw_ablation_4seed.py`).

CHAPTER C

CODE LISTINGS

`detection/graph_builder.py`, `gnn_model.py`, `ensembler.py`, `alert_pipeline.py`,
`drift_monitor.py`, `shap_explainer.py`, `evaluate_gnn.py`. **Nothing in prod imports experiments/.**

CHAPTER D

SUPPLEMENTARY TABLES

Baselines 4-seed (RC-31), externals IDS2018/CTU-13 (RC-29/32), v2 ablation (RC-30), harness costs (RC-14). JSONs: `mw_ablation_4seed.json`, `baselines_4seed.json`, `external_ids2018_multiseed`